Facultatea de Automatică şi Calculatoare, UPB
Departamentul de Calculatoare

Analiza algoritmilor

ŞL.dr.ing. Andrei Mogoş

2020-2021

- Clase de probleme (1) -

Probleme abstracte

Def: O problemă abstractă Q cu date de tip I și rezultate de tip O este o mulțime

$$(Q, I, O) =_{def} \{(\langle Q, i \rangle, r) \mid i \in I \text{ și } r \in Q(i)\}$$

unde perechea $\langle Q, i \rangle$ este o instanță a problemei, iar $Q(i) \subseteq O$ reprezintă mulțimea soluțiilor problemei pentru instanța $\langle Q, i \rangle$.

Def: Problema Q s.n. **de decizie** dacă, pentru o problemă (Q', I, D) dată, este de tipul: "Q' are soluție pentru $i \in I$?".

$(Q, I, \{0,1\}) =_{def} \{(\langle Q, i\rangle, r) \mid i \in I$, și $r \in \{0,1\}$ și $(r = 1 \Leftrightarrow Q'(i) \neq \emptyset)\}$

Obs: Problema de decizie Q studiază existența soluțiilor problemei Q' pentru fiecare valoare $i \in I$.

Def: Problema Q s.n. **de optimizare** dacă selectează soluțiile r ale unei probleme date (Q', I, O) ce optimizează (maximizează sau minimizează) o funcție de cost definită pe $I \times O$.

$(Q, I, O) =_{def} \{(<Q, i>, r) \mid i \in I$ și $r \in \{r' / r' \in Q'(i)$ și $optim(f(i, r'))\}\}$

Obs: Problema de optimizare Q selectează cele mai bune soluții ale unei probleme Q' în raport cu o funcție de cost dată.

Obs: În multe cazuri, problemele de optimizare pot fi formulate ca probleme de decizie sau pot fi rezolvate cu ajutorul unor probleme de decizie

=> În continuare considerăm doar probleme abstracte de decizie.

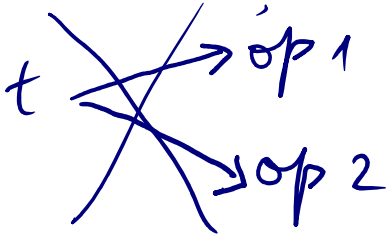
Determinism și nedeterminism

Def: Un algoritm s.n. **determinist** dacă fiecare operație de control sau de prelucrare a datelor are un rezultat unic determinat.

Obs: Într-un algoritm determinist **nu** se execută operații în paralel.

Caracteristica

- caracteristica esențială a unui algoritm determinist : serialitatea algoritmului
- la orice moment de timp din cursul execuției unui algoritm determinist se execută o singură operație
- există o funcție Timp → Operații care descrie execuția algoritmului:



Def: Un algoritm s.n. nedeterminist dacă are cel puțin o operație al cărei rezultat nu este unic definit, ci este o valoare dintr-o mulțime finită de posibilități.

Obs: În cursul execuției unui algoritm nedeterminist există operații care se execută în paralel.

Caracteristici

- caracteristica esențială a unui algoritm nedeterminist: paralelismul nestructional al algoritmului
- execuția unui algoritm nedeterminist conduce la o structură arborescentă de operații. Operațiile de pe o cale sunt executate serial, în timp ce operațiile de pe căi diferite sunt executate în paralel.
- există o submulțime a produsului Timp x Operații care descrie execuția algoritmului: $t \nearrow op_1$, $t \searrow op_2$, ...

Operații necesare pentru construirea unui algoritm nedeterminist

1. choice (A), unde A este o mulțime finită de valori. În momentul execuției operației choice pentru fiecare valoare din A se creează o copie a algoritmului. Multiplicarea presupune construirea copiilor tuturor variabilelor algoritmului cu valorile avute în momentul multiplicării. Copiile algoritmului sunt executate în paralel și independent unele față de celelalte.

2. fail : termină cu insucces execuția copiei algoritmului în care se află. Celelalte copii nu sunt afectate.

3. success : termină cu succes execuția copiei algoritmului în care se află și a tuturor celorlalte copii.

Obs:

- Un algoritm nedeterminist se termină cu succes dacă există o copie a algoritmului care se termină cu succes.
- Un algoritm nedeterminist se termină cu insucces dacă toate copiile algoritmului se termină cu insucces.

Convenție

- Un algoritm nedeterminist terminat cu succes returnează valoarea 1.
- Un algoritm nedeterminist terminat cu insucces returnează valoarea 0.

Def:

Complexitatea unui algoritm nedeterminist este suma complexităților operațiilor din calea cea mai scurtă (adică de complexitate minimă) care termină algoritmul cu succes. Această complexitate s.n. complexitate angelică.

Ex: Să se caute valoarea x în vectorul v cu n elemente.

Algoritm determinist

```
caută(v, n, x)
    for i = 1..n
        if v[i] = x
            print i
            return 1
    return 0
```

$O(n)$

Algoritm nedeterminist

```
N_caută(v, n, x)
    i = choice({1, 2, ..., n})
    if v[i] = x
        print i
        success
    fail
```

$O(1)$

Obs:

Dacă o problemă este rezolvabilă în timpul t_1 cu un algoritm nedeterminist și în timpul t_2 cu un algoritm determinist rezultat prin rescrierea algoritmului nedeterminist atunci $t_1 \leq t_2$.

Obs:

Pe caz general algoritmii nedeterminiști nu se pot rula pe un calculator.

NP - completitudine

Def: Un algoritm s.n. **tractabil** dacă are complexitate polinomială (pentru dimensiunea n a problemei timpul de rezolvare este $O(n^k)$, cu k constant).

Def: O problemă s.n. **tractabilă (deterministă)** dacă poate fi rezolvată folosind un algoritm determinist tractabil.

Def: O problemă s.n. tractabilă nedeterminist dacă poate fi rezolvată folosind un algoritm nedeterminist tractabil.

Def: O problemă s.n. intractabilă dacă toți algoritmii deterministi de rezolvare au complexitate suprapolinomială (pentru dimensiunea problemei n timpul de rezolvare este $\Omega(K^n)$, K constant).

Def: Fie Dec mulțimea problemelor de decizie. Definim următoarele clase de probleme:

$$P = \{ Q \in Dec \mid \text{există cel puțin un algoritm determinist tractabil care rezolvă problema } Q \}$$

$$NP = \{ Q \in Dec \mid \text{există cel puțin un algoritm nedeterminist tractabil care rezolvă problema } Q \}$$

(T) $P \subseteq NP$

Dem

- Fie $Q \in P$ o problemă rezolvabilă printr-un algoritm determinist tractabil Alg cu complexitatea temporală $p(n)$, unde p este un polinom cu valori pozitive.
- Construim un algoritm nedeterminist N_Alg pentru rezolvarea problemei Q.

Algoritmul N_Alg are forma:

- pe o cale (copie a algoritmului) execută operațiile din algoritmul Alg;
- celelalte căi ale algoritmului execută fail.

Algoritmul N_Alg are complexitatea $p(n)$ deoarece singura cale care se poate termina cu succes este cea care execută operațiile din algoritmul Alg.

Rezultă că algoritmul N_Alg are complexitate polinomială. (1)

- N_Alg este algoritm nedeterminist. (2)
- N_Alg rezolvă problema Q deoarece execută aceleași operații ca algoritmul Alg pe calea care se poate termina cu succes. (3)

Din (1), (2) și (3) rezultă că $Q \in NP$.